StudHub — Brief développeur Backend

> **Durée de mission : 2 SEMAINES (10 jours ouvrés)**
> **Stack backend imposé : NestJS + Prisma + PostgreSQL + Redis**
> **Déploiement cible : VPS Linux (Debian 12) via Docker**

🎯 1. Présentation du projet

StudHub est une **application mobile tout-en-un pour la communauté étudiante tunisienne**. C'est un hub qui réunit :

Domaine	Description
🏠 **Immobilier étudiant**	Recherche de studios / colocations / résidences / chambres meublées près des facultés + **espace agent immobilier dédié**
🛒 **Marketplace**	Petites annonces entre étudiants (électronique, meubles, livres, vêtements, vélos…)
💼 **StudServices**	Annuaire de prestataires ciblés étudiants (développeurs, graphistes, réparateurs, profs particuliers…) avec **3 tiers d'abonnement : Gratuit / Standard / Premium**
📰 **Feed social**	Réseau étudiant : publications, événements, clubs, groupes de révision, hackathons, like + commentaires
💬 **Messagerie instantanée**	Chat en temps réel entre étudiants & agents / vendeurs / prestataires
🔐 **2 systèmes d'authentification séparés** : Étudiant (`studhub-session`) + Agent Immobilier (`studhub-agent-session`)	

Deux portails

- **Portail Étudiant** : 6 onglets (Accueil → Feed → Immo → Marketplace → Services → Profil). Accès racine `/tabs/...`
- **Portail Agent Immobilier** : 4 onglets (Dashboard → Gestion biens → Messages → Profil). Accès `/agent-dashboard/...`

📦 2. État actuel du code (frontend)

Ce qui existe déjà — 100% fonctionnel en mock

- ✅ App **Ionic 8 / Angular 20 / Capacitor 8** (compile en APK Android natif)
- ✅ **Toutes les pages et écrans** sont codés et stylés (Tailwind + SCSS)
- ✅ **4 langues** : Français / Anglais / Arabe / Arabe tunisien (Derja) — `src/assets/i18n/`
- ✅ Thème clair / sombre + switch de langue
- ✅ Onboarding Splide + Guard (vue 1 seule fois)
- ✅ Flow auth complet étudiant & agent (register → login → forgot-password → verify-code → reset-password)
- ✅ Tout le typage TypeScript est **déjà écrit** : interfaces `Property` , `Product` , `ServiceProvider` , `StudUser` , `RealEstateAgent` , `ChatMessage` , `Conversation` , `Publisher` , `FeedPost`

Fichiers TypeScript à UTILISER / PARTAGER avec le backend

Ces fichiers contiennent **exactement les schémas de données** — ils font office de **spec contractuelle** :

- [mock-data.service.ts](src/app/services/mock-data.service.ts) → modèles `Property` , `Product` , `ServiceProvider` , `Publisher` + exemples de données
- [auth.service.ts](src/app/services/auth.service.ts) → modèle `StudUser` , logique auth étudiant
- `src/app/services/agent-auth.service.ts` → modèle `RealEstateAgent` , logique auth agent

- [agent-properties.service.ts](src/app/services/agent-properties.service.ts) → modèle `AgentProperty`, CRUD biens
- [messages.service.ts](src/app/services/messages.service.ts) → `Conversation`, `ChatMessage`
- [marketplace.page.ts](src/app/marketplace/marketplace.page.ts) → modèle `Product` (version liste)
- [feed.page.ts](src/app/pages/feed/feed.page.ts) → modèle `FeedPost`

Ce qui n'existe PAS — à construire

- ❌ **Aucun backend** (pas d'URL API, pas de `HttpClient` utilisé sauf pour i18n)
- ❌ **Aucune persistance** — tout est en `localStorage` (auth) ou mémoire vive seule (messages, feed, produits nouvellement créés)
- ❌ **Aucun upload de fichier** — les photos sont traitées en `FileReader` → `dataURL` locale, jamais stockées
- ❌ **Aucun vrai temps réel** — le chat est uniquement local

📐 3. Stack backend IMPOSÉ

> ⚠️ Pas de déviation sur la stack — toute la suite repose dessus.

Couche	Techno	Justification
Framework API	**NestJS 11 (Node.js / TypeScript)**	Même langage TS que le front Angular → partage des DTO / interfaces / enums. Architecture modulaire parfaite pour 4 domaines. Guards / interceptors / WebSocket natifs.
ORM	**Prisma 5+**	Génère le client TS type-safe depuis `schema.prisma`. Migrations BDD zero-douleur. Requêtes CRUD 10x plus rapides à écrire.
Base de données	**PostgreSQL 16**	Relations complexes (user ↔ property ↔ review ↔ conversation ↔ message). JSONB pour metadata. ACID.
Cache + WebSocket Pub/Sub	**Redis 7**	Sessions revoke, rate limit auth, cache listes publiques, et distribution des messages WebSocket si multi-instance.
Upload photos	**Cloudinary** (ou MinIO auto-hébergé sur le VPS)	CDN + transformations d'images. Simple, gratuit en tier MVP. Les URLs Cloudinary sont stockées en BDD.
Auth	**JWT (access token 15min + refresh token 7j)**	Access token en header `Authorization: Bearer`. Refresh token en cookie `httpOnly` + `secure` en prod.
Validation	`class-validator` + `class-transformer` (DTOs Nest)	Validation d'entrée identique à ce qui existe côté front Angular.
Conteneurisation	**Docker + docker-compose**	3 services : `api` (Nest), `db` (PostgreSQL), `redis` (Redis). Déploiement identique en local / dev / VPS.
Reverse proxy + SSL	**Caddy 2** sur le VPS	HTTPS automatique Let's Encrypt, gzip, headers sécurité.

Arborescence à créer

```
studhub/
├── backend/                          ← DOSSIER À CRÉER À CÔTÉ DE src/
│   ├── prisma/
│   │   ├── schema.prisma             ← modèles de BDD (inspirés de mock-data.service.ts)
│   │   └── migrations/
│   ├── src/
│   │   ├── main.ts                   ← bootstrap Nest : app.setGlobalPrefix('api') + CORS + ValidationPipe
│   │   ├── app.module.ts             ← imports : PrismaModule, ConfigModule, JwtModule, ThrottlerModule
│   │   ├── common/
│   │   │   ├── guards/               ← JwtAuthGuard, RolesGuard('student' | 'agent')
│   │   │   ├── decorators/           ← @CurrentUser(), @Roles()
│   │   │   ├── filters/              ← PrismaClientExceptionFilter (traduit erreurs Prisma en HTTP)
│   │   │   └── interceptors/         ← ResponseInterceptor, LoggingInterceptor
│   │   ├── prisma/
│   │   │   ├── prisma.module.ts
│   │   │   └── prisma.service.ts     ← extends PrismaClient + onModuleInit
│   │   ├── config/
│   │   │   └── configuration.ts      ← process.env : DATABASE_URL, REDIS_URL, JWT_SECRET, CLOUDINARY_, FRONTEND_URL
│   │   └── modules/
│   │       ├── auth/                 ← 🔐 étudiant + agent (2 stratégies JWT distinctes ou 1 avec champ role)
│   │       │   ├── auth.module.ts
│   │       │   ├── auth.service.ts       ← register, login, refresh, forgotPassword, sendCode, verifyCode, resetPassword
│   │       │   ├── auth.controller.ts    ← POST /api/auth/ + POST /api/agent-auth/
│   │       │   ├── dto/                   ← RegisterDto, LoginDto, ForgotDto, VerifyCodeDto, ResetDto…
│   │       │   └── strategies/            ← JwtStrategy + éventuellement LocalStrategy
│   │       ├── properties/           ← 🏠 logements immobiliers
│   │       │   ├── properties.module.ts
│   │       │   ├── properties.service.ts  ← CRUD + pagination + filtres (ville, prix, surface, catégorie, meublé…)
│   │       │   ├── properties.controller.ts  ← GET public + CRUD protégé role=agent
│   │       │   └── dto/                   ← CreatePropertyDto, UpdatePropertyDto, PropertyFilterDto
│   │       ├── marketplace/          ← 🛒 produits du marketplace
│   │       │   ├── marketplace.module.ts
│   │       │   ├── products.service.ts
│   │       │   ├── products.controller.ts  ← GET public / POST/PUT/DELETE protégé role=student (propriétaire)
│   │       │   └── dto/
│   │       ├── services/             ← 💼 prestataires StudServices
│   │       │   ├── services.module.ts
│   │       │   ├── providers.service.ts     ← tiers Gratuit/Standard/Premium, recherche par catégorie/ville
│   │       │   ├── providers.controller.ts
│   │       │   └── dto/
│   │       ├── feed/                 ← 📰 posts sociaux
│   │       │   ├── feed.module.ts
│   │       │   ├── posts.service.ts         ← create, list paginée, like/unlike, addComment
│   │       │   ├── posts.controller.ts
│   │       │   └── dto/
│   │       ├── chat/                 ← 💬 messagerie
│   │       │   ├── chat.module.ts
│   │       │   ├── chat.gateway.ts          ← @WebSocketGateway : joinRoom, sendMessage, typing…
│   │       │   ├── conversations.service.ts ← CRUD conversations + count unread
│   │       │   ├── messages.service.ts      ← persistence messages en BDD
│   │       │   └── dto/
│   │       ├── reviews/              ← ⭐ avis (optionnel mais prévoir)
│   │       └── upload/               ← 🖼️ upload photos (Cloudinary ou MinIO)
│   │           ├── upload.module.ts
│   │           ├── upload.service.ts       ← upload via buffer stream, retourne secure_url
│   │           └── upload.controller.ts    ← POST /api/upload → multipart/form-data (files[])
│   ├── test/
│   ├── .env.example                ← VARIABLES D'ENVIRONNEMENT À REMPLIR (copier en .env)
│   ├── Dockerfile                  ← build image Node API (multi-stage : builder + production alpine)
│   ├── docker-compose.yml          ← services: api + db + redis
│   └── package.json                ← @nestjs/cli, prisma, @prisma/client, @nestjs/jwt, @nestjs/passport, @nestjs/websockets, @nestjs/throttler, clou
│
└── src/                             ← FRONTEND EXISTANT (Ionic/Angular) — ne PAS toucher sauf section 6
```

Variables d'environnement (.env.example)

```
=== APP ===

NODE_ENV=development
PORT=3000
FRONTEND_URL=http://localhost:4200

=== SECRETS ===

JWT_ACCESS_SECRET=studhub_access_change_me_in_prod_xxxxxxxxxxxxxxx
JWT_REFRESH_SECRET=studhub_refresh_change_me_in_prod_yyyyyyyyyyyyyyy
JWT_ACCESS_TTL=900
JWT_REFRESH_TTL=604800

=== BDD ===

DATABASE_URL=postgresql://studhub:studhub_password@localhost:5432/studhub?schema=public

=== REDIS ===

REDIS_URL=redis://localhost:6379

=== CLOUDINARY ===

CLOUDINARY_CLOUD_NAME=xxxx
CLOUDINARY_API_KEY=xxxx
CLOUDINARY_API_SECRET=xxxx

=== MAIL (optionnel MVP : stub console en dev) ===

SMTP_HOST=smtp.example.com

SMTP_PORT=587

SMTP_USER=noreply@studhub.tn

# SMTP_PASS=xxx
```

🔗 4. Liste des endpoints à implémenter (spec)

> *Préfixe global :* `/api`
> *Format de réponse uniforme :* `{ "success": true, "data": ..., "meta": { ...pagination... } }` *(via* `ResponseInterceptor` *)*
> *Erreurs :* `{ "success": false, "error": "CODE_ERR", "message": "...", "details": [...] }`
> *Pagination par défaut :* `?page=1&limit=20`

🔐 Auth Étudiant (`/api/auth/`)

Méthode	Route	Body	Rôle	Description
POST	`/auth/register`	`{fullName, email, password, confirmPassword, phone?, university?}`	public	Inscription + renvoie access+refresh
POST	`/auth/login`	`{email, password}`	public	Connexion → access+refresh
POST	`/auth/refresh`	(cookie refreshToken)	public	Nouvel access token
POST	`/auth/logout`	—	student	Invalide refresh côté Redis
POST	`/auth/forgot-password`	`{email}`	public	Génère code 6 chiffres → stocke Redis (10 min). **En dev : renvoyer le code dans la réponse.** En prod → envoyer par email.
POST	`/auth/verify-code`	`{email, code}`	public	Vérifie code
POST	`/auth/reset-password`	`{email, code, newPassword}`	public	Réinitialise MDP
GET	`/auth/me`	—	student	Renvoie profil étudiant connecté
PATCH	`/auth/me`	`{fullName?, phone?, university?}`	student	Modifier profil

🔐 Auth Agent (`/api/agent-auth/`)

Même flow exact mais modèle `RealEstateAgent` *avec* `agencyName` + `role='agent'`

🏠 Logements (`/api/properties`) — Inspirations modèles [mock-data.service.ts](src/app/services/mock-data.service.ts#L13-L31)

Méthode	Route	Query params	Rôle
GET	`/properties`	`?city=&minPrice=&maxPrice=&category=Studios`	Colocati
GET	`/properties/:id`	—	public
POST	`/properties`	multipart : `{title,category,city,address,price,surface,rooms,floor,furnished,availableFrom,description,amenities[],photos[]}`	**agent**
PATCH	`/properties/:id`	(champs partielles)	**agent** (propriét
DELETE	`/properties/:id`	—	**agent** (propriét
GET	`/agent/me/properties`	`?page=`	**agent**

🛒 Marketplace Produits (`/api/products`)

Inspiration : [mock-data.service.ts](src/app/services/mock-data.service.ts#L33-L50) + [marketplace.page.ts](src/app/marketplace/marketplace.page.ts#L5-L16)

- GET `/products` (filtres `category` , `condition` , `minPrice` , `city` , `search`)
- GET `/products/:id`
- POST `/products` (student connecté)
- PATCH `/products/:id` (propriétaire)
- DELETE `/products/:id` (propriétaire)

💼 Prestataires StudServices (`/api/providers`)

Inspiration : [mock-data.service.ts](src/app/services/mock-data.service.ts#L52-L73)

- GET `/providers` (filtres `category` , `tier` , `city` , `search`)
- GET `/providers/:id`
- POST `/providers` (student/agent avec `role=provider` + `tier`)
- PATCH `/providers/:id` (propriétaire)

📰 Feed Posts (`/api/posts`)

Inspiration : [feed.page.ts](src/app/pages/feed/feed.page.ts#L3-L15)

- GET `/posts` (?search= & page= & limit=)
- POST `/posts` (student) → `{description, imageUrl?}`
- POST `/posts/:id/like` (toggle like)
- GET `/posts/:id/comments`
- POST `/posts/:id/comments` → `{text}`

💬 Messagerie

API REST :

- GET `/conversations` → liste de mes convs + unreadCount

- *GET `/conversations/:id/messages?before=` → messages paginés par conversation*
- *POST `/conversations` → `{withUserId, withRole, context?}` initier une conv avec user/agent/prestataire/vendeur*

WebSocket (`ws://host/ws/chat` — auth via JWT dans query ou header) :

- *Événements client : `join(conversationId)` , `message({conversationId, text})` , `typing({conversationId, isTyping})`*
- *Événements serveur broadcast : `new_message` , `typing` , `conversation_updated`*
- ***Distribution via Redis Pub/Sub** pour supporter scalabilité multi-instance (obligatoire pour VPS)*

🖼️ Upload

- *POST `/upload` (multipart form-data `file`) → `{url, publicId, width, height}` — rôle authentifié student OU agent. Accepte jpg/png/webp. Taille max 5Mo. Compression auto côté Cloudinary.*

🚀 5. Déploiement VPS — Spécifications & procédure

Matériel VPS (minimum MVP Tunisie)

- ***OS : Debian 12 Bookworm** (x86_64)*
- ***vCPU :** 2 vCPUs*
- ***RAM :** 4 Go minimum (Nest + Postgres + Redis + Caddy dans Docker tient très bien dedans)*
- ***Stockage :** 80 Go SSD*
- ***Hébergeur recommandé : Scaleway DEV1-S** (€5/mois) ou **DigitalOcean Basic 4GB** ($12/mois) ou **OVH Starter** — choisir DC en EU (Paris/Frankfurt/Amsterdam) pour latence décente Tunisie.*
- ***IP statique** + **PTR inversé** correct.*
- ***Nom de domaine :** ex `studhub.tn` avec 2 sous-domaines A :*
- `app.studhub.tn` *→ frontend Ionic (build www/ servi par Caddy ou séparément Netlify/Vercel)*
- `api.studhub.tn` *→ backend Nest + Docker*
- *Optionnel : `minio.studhub.tn` si pas Cloudinary*

Outils d'exploitation (à installer sur le VPS)

- ***Docker 27+** + **Docker Compose v2** (déploiement par conteneurs)*
- ***Caddy 2** en hôte (ou en conteneur) → HTTPS auto + reverse proxy vers `localhost:3000`*
- ***UFW firewall** ouvert sur : `22 (ssh limité)` , `80` , `443` → tout le reste fermé*
- ***fail2ban** pour bannir brute force SSH*
- *Monitoring léger : `netdata` en conteneur (port 19999 limité IP)*

Pipeline de déploiement

1. ***Build local** de l'API : image Docker multi-stage (`node:22-alpine builder → node:22-alpine slim`)*
2. ***Tag** image en `registry.studhub.tn/api:latest` (ou utiliser Docker Hub) OR utiliser **CapRover / Coolify** sur le VPS pour pull + build direct depuis un repo Git privé*
3. ***Sur VPS :***

```
cd /opt/studhub
docker compose pull
docker compose up -d --remove-orphans
docker compose exec api npx prisma migrate deploy   # applique migrations en BDD
```

1. ***Caddyfile VPS (/etc/caddy/Caddyfile) :***

```
api.studhub.tn {
  reverse_proxy localhost:3000
  encode gzip zstd
  header {
    X-Frame-Options DENY
    X-Content-Type-Options nosniff
    Referrer-Policy strict-origin-when-cross-origin
    Permissions-Policy "geolocation=(), microphone=(), camera=()"
  }
}
```

Sauvegardes

- *Dump PostgreSQL **quotidien** à 3h du matin via cron : `pg_dump` + upload sur bucket S3 compatible (Scaleway Object Storage 750GB/mois gratuit)*
- *Volume Docker `postgres_data` dans `/var/lib/docker/volumes/...` — sauvegardé*
- *Redis : backup RDB AOF + inclus dans backup complet VPS*

Health check

- *GET `/api/health` → retourne `{status:'ok', db:true, redis:true}` → **obligatoire** pour uptime monitoring (UptimeRobot gratuit sur cette URL)*

🗓️ *6. Planning 2 SEMAINES = 10 jours ouvrés*

> ⚠️ ***Périmètre de la mission = Backend + déploiement VPS + connexion front**. Pas de refonte frontend UX, pas d'app mobile native supplémentaire.*

🟢 *Semaine 1 — Fondations + modules critiques (JOUR 1 → JOUR 5)*

Jour	*Livrable attendu*
J1	*Initialisation projet `backend/` : NestJS CLI, Prisma, Docker Compose (api + db + redis), Caddyfile dev local, `.env.example`, fichier README backend. **Schéma Prisma entier** écrit et validé (toutes les tables du projet). `prisma migrate dev --name init` → 1ère migration créée. Lancer l'app hello world sur `http://localhost:3000/api/health`. **Partager repo Git** + push premier commit.*
J2	*Module **Auth** 100% fonctionnel : JWT access+refresh, DTOs validés, `RolesGuard`, endpoints `/api/auth/` et `/api/agent-auth/`. **Endpoints testés via Postman / Bruno** : register, login, refresh, forgot, verify, reset. 2 rôles créés : `student` et `agent`. Revocation refresh token sur Redis fonctionnel.*
J3	*Module **Properties** (Immobilier) : CRUD BDD + filtres + pagination. Endpoints testés via collection Bruno. Test agent créé bien → étudiant voit bien dans liste. `403 Forbidden` si étudiant tente POST/PATCH/DELETE. Upload photos connecté Cloudinary dans module `upload` + POST `/properties` accepte multipart photos.*
J4	*Module **Marketplace (Products)** + **StudServices (Providers)** : CRUD + filtres + recherche. Modules **Upload** (Cloudinary) finalisé. Module **Feed Posts** : CRUD posts + toggle like + comments.*
J5	*Module **Chat** : `ConversationsService` + `MessagesService` (BDD) + **WebSocket Gateway Nest** fonctionnelle en local (test via websocat ou page test). Redis Pub/Sub opérationnel. REST conversations + messages OK.*

🔵 *Semaine 2 — Intégration + déploiement + livraison (JOUR 6 → JOUR 10)*

Jour	*Livrable attendu*
J6	***Intégration FRONT ↔ BACK** : Modifier le frontend Ionic existant (uniquement les services, **NE PAS TOUCHER aux templates HTML ni au style**). Créer un nouveau dossier `src/app/core/api/` avec : `api.interceptor.ts` (ajoute Authorization Bearer JWT + refresh automatique 401), + nouveaux services Angular `properties-api.service.ts` , `products-api.service.ts` , `providers-api.service.ts` , `posts-api.service.ts` , `chat-api.service.ts` , `auth-api.service.ts` . Supprimer les mocks dans les pages et remplacer par appels API. Fichier `environment.ts` mis à jour avec `{ apiUrl: 'http://localhost:3000/api', wsUrl: 'ws://localhost:3000/ws/chat' }` .*
J7	***Provisionnement VPS Debian 12** : SSH user + sudo, Docker+Compose, Caddy, UFW, fail2ban configurés. DNS `api.studhub.tn` pointe sur IP VPS. HTTPS Let's Encrypt fonctionnel. **Premier déploiement Docker Compose sur le VPS** réussi : `/api/health` accessible en HTTPS public. Base de données persistée dans volume Docker.*
J8	***End-to-end tests manuels** sur 2 appareils : scénarios complets. Liste de contrôle : (1) Inscription étudiant → (2) Login → (3) Recherche logements avec filtres → (4) Détail bien → (5) Inscription agent → (6) Agent publie bien avec photos → (7) Étudiant voit apparaître le bien dans la liste → (8) Étudiant et agent ouvrent conversation → (9) Chat temps réel bidirectionnel fonctionne. Bug fixes.*
J9	***Optimisation + hardening** : Rate limiting (ThrottlerModule Nest : 100 req/minute par IP sur login/forgot pour bloquer brute force). Headers sécurité CSP. `bcrypt` cost 12 minimum pour les mots de passe. CORS restreint à `FRONTEND_URL` + `app.studhub.tn` en prod. Documentation OpenAPI Swagger Nest (endpoint `/api/docs`) avec tous les groupes endpoints. **Backup script PostgreSQL + cron activé sur VPS.***
J10 — JOUR DE LIVRAISON	***Livrable final + transfert**. (1) Repos Git up-to-date : frontend+backend 100% synchronisés. (2) README unique à la racine qui explique "comment lancer front en local, comment lancer back en local docker, comment déployer sur VPS". (3) Collection Bruno / Postman complète (tous les endpoints + auth). (4) Dashboard de monitoring UptimeRobot sur `/api/health` . (5) Démo live 30 min en appel où on déroule tous les scénarios du J8 en PROD sur `app.studhub.tn` / `api.studhub.tn` .*

🚨 *Engagements qualité OBLIGATOIRES*

- ❌ *Pas de `// TODO` / `console.log` / `any` laissés dans le code livré*
- ✅ ***Toutes les requêtes en écriture** passent par un DTO avec `class-validator`*
- ✅ *HTTP status corrects : 200 / 201 / 204 / 400 / 401 / 403 / 404 / 409 / 422 / 500*
- ✅ *Mots de passe **jamais retournés** par un endpoint (même pas à l'admin)*
- ✅ `bcrypt` *pour tous les hash (JAMAIS md5/sha1)*
- ✅ *Les secrets (JWT_, CLOUDINARY_) **sont dans .env uniquement**, jamais commités sur Git*
- ✅ `.gitignore` *backend : `.env` , `node_modules/` , `dist/` , `.log`*
- ✅ Logs JSON structurés côté API (Nest PinoLogger recommandé)
- ✅ **Migrations Prisma versionnées**, jamais `prisma db push` en prod
- ✅ Frontend continue à compiler sans erreur `ng build` après modifications J6

🧪 7. Comment tester le front actuel (lecture seule — pour le dev backend)

Prérequis : Node 20+, npm 10+

```
git clone <LE_REPO>
cd studhub
```

Install front

```
npm install
```

Lancer front Angular/Ionic

```
npm start
# → Ouvre http://localhost:4200
```

Depuis Chrome : **F12** → **Ctrl+Shift+M** → **iPhone 14** pour avoir la vue mobile. Tout est en mock. Le développeur backend peut naviguer librement et inspecter les données mockées pour comprendre chaque page.

📝 8. Notes utiles

Devise

Tous les prix sont en **Dinar tunisien (DT)** — champs `price: number` entier. Côté affichage le front ajoute `/mois` pour l'immobilier.

Numéros de téléphone

Format tunisien `22 456 789` (8 chiffres). Indicatif international **+216** — voir helpers `telLink()` / `whatsappLink()` dans [mock-data.service.ts](src/app/services/mock-data.service.ts#L682-L688).

Tiers prestataires

Classes CSS à retourner en string pour les badges (couleurs d'origine dans le front, mais backend stocke seulement l'enum) : `tier ∈ ('Gratuit' | 'Standard' | 'Premium')`

Filtres logement

Catégories d'immobilier existantes dans le front mock : `'Studios' | 'Colocations' | 'Résidences' | 'Chambres'` → **utiliser ces valeurs EXACTES en enum Prisma** sinon le casse.

Étapes suivantes pour toi, commanditaire

- Commander le VPS + nom de domaine avant le J7 max
- Créer compte Cloudinary gratuit avant J3
- Donner accès SSH au développeur avant J7

Fin du brief. Toute déviation de stack, de planning ou de scope doit être validée PAR ÉCRIT avant de commencer.